

Dynamic Programming & Reinforcement Learning

Function Approximation and Least-Squares DP

Instructor: Chuanhao Li

October 27, 2026

Content for Today

1. Represent values with shared features 6–14

From a value table to a small parameter vector

2. Fit parameters by least squares 15–29

Targets, normal equations, and regularization

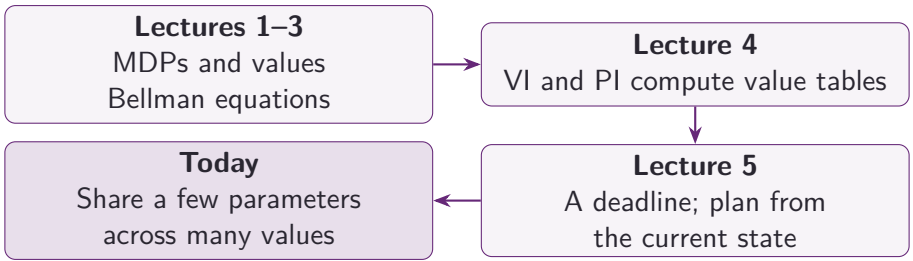
3. Put regression inside backward DP 30–48

Least-squares value iteration and a complete job example

4. Understand errors and the next step 49–62

What approximation changes; from model targets to data

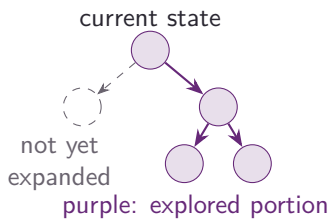
The course so far: the same Bellman idea, new obstacles



Later: use experience when exact reward and transition calculations are unavailable.

Search and approximation address different costs

Lecture 5: where to compute?
Search concentrates effort near the current state.



Today: how to store and share values?
A function uses shared parameters to predict values at many inputs.

$$\text{state–action pair} \longmapsto \text{estimated value}.$$

Changing one parameter can change many predictions.
An approximator can also supply leaf values to a search tree.

Today we keep the model known. Approximation is useful even before learning from experience.

Keep the finite-horizon setting from Lecture 5

There are H decisions, at stages $t = 1, \ldots, H$.

$$G_t = \sum_{s=t}^{H} R_s + g(X_{H+1}), \qquad V^*_{H+1} = g.$$

- Finite state set $\mathcal{X}$, possibly very large; finite nonempty feasible sets $\mathcal{A}(x)$.
- Known mean reward $r(x, a)$ and transition probabilities $\mathcal{P}(x' \mid x, a)$; the same rules at every stage.
- Bounded rewards and known terminal payoff g; no discount factor in this finite sum.

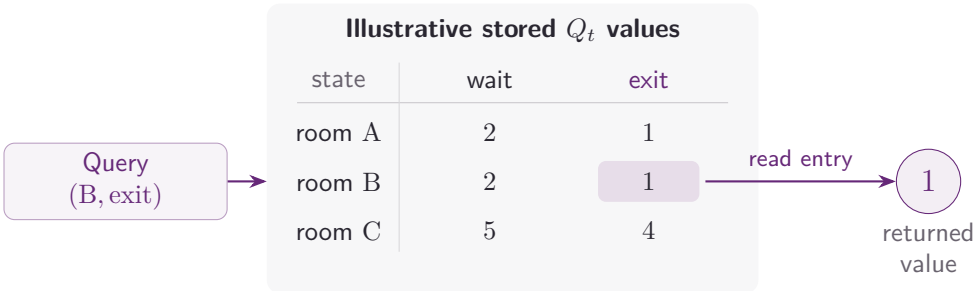
Values and decision rules keep the stage index: V_t, Q_t, π_t. The terminal payoff is paid once, with no action at $H + 1$.

Representing Values with Features



So far: store values in a table and look them up

A V_t table has one entry per state; a Q_t table has one per feasible pair.

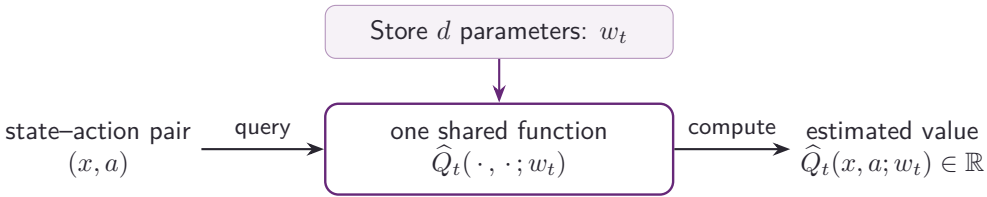


All six pairs are feasible. Room and action names are **labels used as addresses**.

Large state spaces require large tables. How else could we represent their values?

Function approximation: represent values with parameters

Choose a family of functions. A stored parameter vector $w_t \in \mathbb{R}^d$ selects one function to **estimate values at all feasible pairs**.



The same parameters are reused for every input.

d is the number of parameters. A smaller representation may restrict which values we can fit.

Write $\widehat{Q}_t(x, a)$ when w_t is fixed. Next: how should a function use (x, a)?

A feature map turns a state–action pair into numbers

A **feature map** assigns d numerical descriptors to each feasible pair:

$$\phi : \{(x,a) : x \in \mathcal{X},\ a \in \mathcal{A}(x)\} \longrightarrow \mathbb{R}^d, \qquad \phi(x,a) = \begin{bmatrix} \phi_1(x,a) \\ \vdots \\ \phi_d(x,a) \end{bmatrix}.$$

Each ϕ_j is a chosen real-valued function of the input. A **linear value approximator** combines these descriptors with weights:

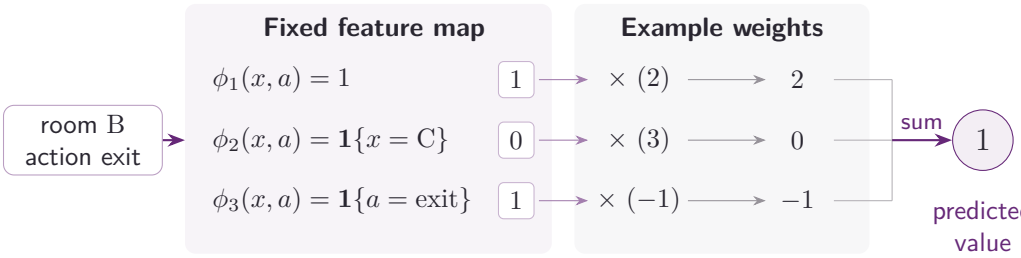
$$\boxed{\widehat{Q}_t(x,a) = \phi(x,a)^\top w_t = \sum_{j=1}^{d} \phi_j(x,a) w_{t,j}.}$$

$w_{t,j}$ is weight j. Today, ϕ is fixed and the weights vary by stage.

Linear in the weights: the features can be nonlinear functions or tests of named categories.

From named inputs to features, then to a prediction

Use rooms $\{\mathrm{A}, \mathrm{B}, \mathrm{C}\}$ and actions $\{\mathrm{wait}, \mathrm{exit}\}$. Choose the three features below; $\mathbf{1}\{\cdot\}$ is 1 if true and 0 otherwise.



$$\phi(\mathrm{B}, \mathrm{exit}) = (1, 0, 1)^\top, \quad w_t = (2, 3, -1)^\top, \quad \widehat{Q}_t(\mathrm{B}, \mathrm{exit}) = 2 + 0 - 1 = 1.$$

The map creates numerical features; the room and action labels are never multiplied.

One-hot features make the dot product a table lookup

A **one-hot vector** has one 1 and zeros elsewhere: use one coordinate per feasible pair. For the same query $(\mathrm{B}, \mathrm{exit})$, activate only its coordinate:

	Coordinate / pair	$\phi(\mathrm{B}, \mathrm{exit})$		w_t: the stored table		**Product**
1	$(\mathrm{A}, \mathrm{wait})$	0	$\times$	2	$=$	0
2	$(\mathrm{A}, \mathrm{exit})$	0	$\times$	1	$=$	0
3	$(\mathrm{B}, \mathrm{wait})$	0	$\times$	2	$=$	0
4	$(\mathrm{B}, \mathrm{exit})$	1	$\times$	1	$=$	1
5	$(\mathrm{C}, \mathrm{wait})$	0	$\times$	5	$=$	0
6	$(\mathrm{C}, \mathrm{exit})$	0	$\times$	4	$=$	0

Sum $= 1$: the query selects exactly one stored entry.

The dot product returns exactly the same value as lookup. Each weight stores one independent table entry.

Why one-hot features can represent any value table

Number of feasible pairs: $N = \sum_{x \in \mathcal{X}} |\mathcal{A}(x)|$. Use one coordinate per pair.

With S states and A available actions at each state, $N = SA$.

Here, 3 rooms and 2 actions give $N = 6$. Query: $(\mathrm{B}, \mathrm{exit})$.

	$(\mathrm{A}, \mathrm{wait})$	$(\mathrm{A}, \mathrm{exit})$	$(\mathrm{B}, \mathrm{wait})$	$(\mathrm{B}, \mathrm{exit})$	$(\mathrm{C}, \mathrm{wait})$	$(\mathrm{C}, \mathrm{exit})$
one-hot	0	0	0	1	0	0
weight	$w_{t,1}$	$w_{t,2}$	$w_{t,3}$	$w_{t,4}$	$w_{t,5}$	$w_{t,6}$

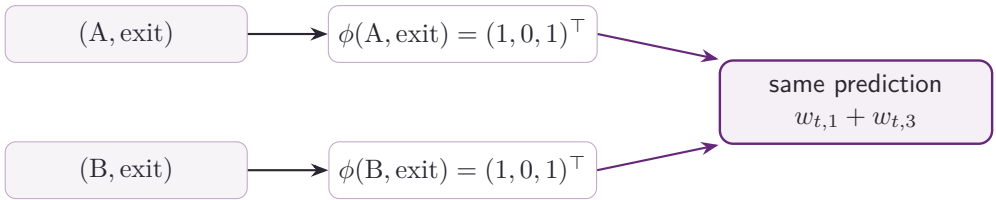
$$\widehat{Q}_t(\mathrm{B}, \mathrm{exit}) = 0 + 0 + 0 + 1 \cdot w_{t,4} + 0 + 0 = w_{t,4}.$$

Every pair works the same way: set each weight to its table entry.

One-hot uses $d = N$ weights: no compression. With $d < N$, a fixed linear approximator cannot represent every table.

Shared features make different pairs use the same weights

Return to the three-feature map $\phi(x, a) = (1, \mathbf{1}\{x = \mathrm{C}\}, \mathbf{1}\{a = \mathrm{exit}\})^\top$.



These features do not distinguish room A from room B.

Changing $w_{t,3}$ changes **both predictions** by the same amount.

One-hot gives different pairs separate coordinates; these features tie some predictions together.

Shared features can prevent an exact fit

The same features force the same prediction, for **every** choice of weights:

$$\phi(x, a) = \phi(\widetilde{x}, \widetilde{a}) \quad \Longrightarrow \quad \widehat{Q}_t(x, a) = \widehat{Q}_t(\widetilde{x}, \widetilde{a}).$$

Useful sharing

One weight update can improve estimates at several inputs.
We store shared weights instead of independent values for every pair.

A possible conflict

Suppose the desired targets were

$$(\mathrm{A}, \mathrm{exit}) \mapsto 1, \qquad (\mathrm{B}, \mathrm{exit}) \mapsto 3.$$

Their identical features force equal predictions: these targets cannot both be fitted exactly.

Next: for the chosen features, how should we choose the weights?

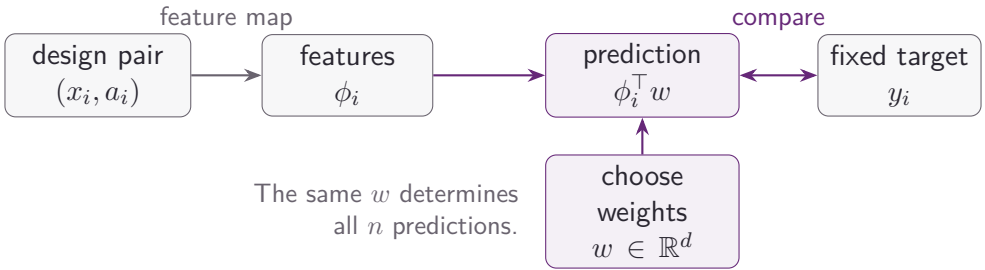
Fitting Values by Least Squares



First solve a simpler problem: fit given targets

Fix one stage and omit t. The data are n design pairs and fixed labels:

$$\{(x_i, a_i, y_i)\}_{i=1}^{n}, \qquad \phi_i := \phi(x_i, a_i) \in \mathbb{R}^d.$$



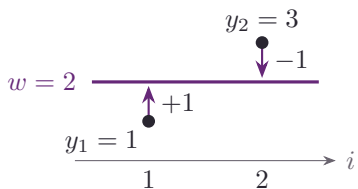
Adjust w and its predictions; keep the features and targets fixed.

The targets need not be optimal values. Later, DP will construct them.

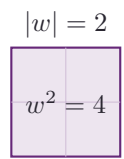
Choose weights by balancing fit and size

$$\widehat{w} \in \arg\min_{w \in \mathbb{R}^d} \left\{ \sum_{i=1}^{n} (\phi_i^\top w - y_i)^2 + \lambda \|w\|_2^2 \right\}, \qquad \lambda \geq 0.$$

Illustration: $\phi_1 = \phi_2 = 1$, $y_1 = 1$, $y_2 = 3$; try $w = 2$.



fit error: $1^2 + (-1)^2 = 2$



penalty: $\lambda w^2 = 4\lambda$

$\lambda = 0$: ordinary least squares. $\lambda > 0$: ridge regression.

We use the **sum** of squared errors; $\|w\|_2^2 = \sum_j w_j^2$.

Stack the predictions into one linear system

Write $\phi_i = (\phi_{i1}, \ldots, \phi_{id})^\top$. Each matrix row is one pair; each column is one feature.

$$\underbrace{\begin{bmatrix} \phi_{11} & \cdots & \phi_{1d} \\ \vdots & & \vdots \\ \phi_{i1} & \cdots & \phi_{id} \\ \vdots & & \vdots \\ \phi_{n1} & \cdots & \phi_{nd} \end{bmatrix}}_{\Phi \in \mathbb{R}^{n \times d}} \underbrace{\begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}}_{w \in \mathbb{R}^d} = \underbrace{\begin{bmatrix} \phi_1^\top w \\ \vdots \\ \phi_i^\top w \\ \vdots \\ \phi_n^\top w \end{bmatrix}}_{\text{prediction vector } \Phi w} \qquad \underbrace{\begin{bmatrix} y_1 \\ \vdots \\ y_i \\ \vdots \\ y_n \end{bmatrix}}_{\text{target vector } y} .$$

Row i: $\phi_i^\top w - y_i = \sum_{j=1}^{d} \phi_{ij} w_j - y_i.$

$$\boxed{L(w) := \|\Phi w - y\|_2^2 + \lambda \|w\|_2^2.}$$

OLS minimizes residual length; ridge also penalizes weight size.

Differentiate to obtain the normal equations

Expand the quadratic before differentiating:

$$L(w) = w^\top \Phi^\top \Phi w - 2w^\top \Phi^\top y + y^\top y + \lambda w^\top w.$$

Because $\Phi^\top \Phi$ is symmetric,

$$\nabla L(w) = 2\Phi^\top \Phi w - 2\Phi^\top y + 2\lambda w.$$

Setting this gradient to zero gives

$$\boxed{(\Phi^\top \Phi + \lambda I_d)\widehat{w} = \Phi^\top y.}$$

These are the **normal equations**. A zero gradient gives a candidate. Why does this candidate minimize the loss?

Why the normal equations give a minimum

Suppose $\widehat{w}$ satisfies the normal equations. Compare it with any other weights w by expanding both squares:

$$L(w) - L(\widehat{w}) = \|\Phi(w - \widehat{w})\|_2^2 + \lambda\|w - \widehat{w}\|_2^2 + 2(w - \widehat{w})^\top \underbrace{[\Phi^\top(\Phi\widehat{w} - y) + \lambda\widehat{w}]}_{=0 \text{ by the normal equations}} .$$

The cross term vanishes. Since $\lambda \geq 0$, the remaining terms are nonnegative:

$$\boxed{L(w) \geq L(\widehat{w}) \qquad \text{for every } w \in \mathbb{R}^d.}$$

Thus $\widehat{w}$ is a **global minimizer**: no weights give a smaller loss. When is the minimizer unique?

Ridge makes the coefficient matrix invertible

Define the $d \times d$ matrix

$$\Lambda := \Phi^\top \Phi + \lambda I_d = \sum_{i=1}^{n} \phi_i \phi_i^\top + \lambda I_d.$$

For any nonzero $v \in \mathbb{R}^d$, if $\lambda > 0$,

$$v^\top \Lambda v = \|\Phi v\|_2^2 + \lambda \|v\|_2^2 > 0.$$

Thus Λ is positive definite and

$$\boxed{\widehat{w} = \Lambda^{-1} \Phi^\top y = \Lambda^{-1} \sum_{i=1}^{n} \phi_i y_i.}$$

For $\lambda = 0$, the inverse formula requires Φ to have full column rank. In code, solve the linear system rather than explicitly forming the inverse.

A complete fit with one shared feature

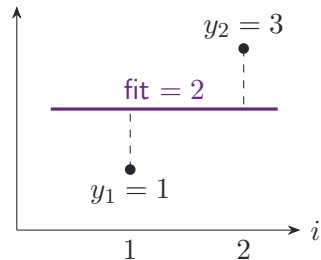
At two design pairs, use the same feature: $\phi_1 = \phi_2 = 1$. The targets are $y_1 = 1$, $y_2 = 3$, and $\lambda = 0$.
Both predictions equal the scalar w:

$$L(w) = (w-1)^2 + (w-3)^2.$$

Use $\widehat{w} = (\sum_i \phi_i^2)^{-1} \sum_i \phi_i y_i$:

$$\widehat{w} = \frac{1 \cdot 1 + 1 \cdot 3}{1^2 + 1^2} = 2.$$

Residual means **prediction minus target**: here 1 and -1. The squared error is 2.



Check: what changes when we add ridge?

Keep the same data, now with $\lambda = 2$:

$$\phi_1 = \phi_2 = 1, \qquad y_1 = 1, \qquad y_2 = 3.$$

The scalar normal equation is

$$\left(\lambda + \sum_{i=1}^{2} \phi_i^2\right) \widehat{w} = \sum_{i=1}^{2} \phi_i y_i.$$

1. Compute the coefficient on the left and the target sum on the right.
2. What are $\widehat{w}$ and the two fitted predictions?
3. Compared with the unregularized fit $\widehat{w} = 2$, does the training error decrease?

Answer: squared fit error + weight penalty

$\phi_1 = \phi_2 = 1$, $y_1 = 1$, $y_2 = 3$, $\lambda = 2$: both predictions equal w.

$$(2 + 1^2 + 1^2)\widehat{w} = 1 + 3 \quad \Longrightarrow \quad 4\widehat{w} = 4 \quad \Longrightarrow \quad \boxed{\widehat{w} = 1.}$$

$$L(w) = \underbrace{(w-1)^2 + (w-3)^2}_{\text{squared fit error}} + \underbrace{2w^2}_{\text{weight penalty } (\lambda w^2)} .$$

w	■ **Squared fit error**	□ **Weight penalty**	**Total loss**
2	$(2-1)^2 + (2-3)^2 = 2$	$2 \cdot 2^2 = 8$	2 | 8 $= 10$
1	$(1-1)^2 + (1-3)^2 = 4$	$2 \cdot 1^2 = 2$	4 | 2 $= 6$

The fit error rises by 2, but the penalty falls by 6: total loss falls by 4.

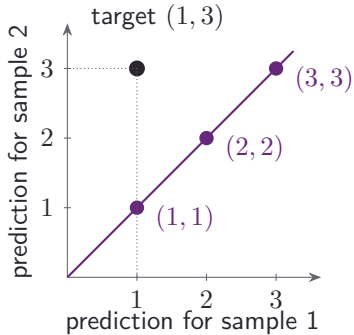
One point records two predictions

Same example: $\phi_1 = \phi_2 = 1$, with target vector $y = (1, 3)^\top$.

Both predictions share one scalar w:

$$\Phi = (1, 1)^\top, \qquad \Phi w = (w, w)^\top.$$

w	Prediction 1	Prediction 2
1	1	1
2	2	2
3	3	3

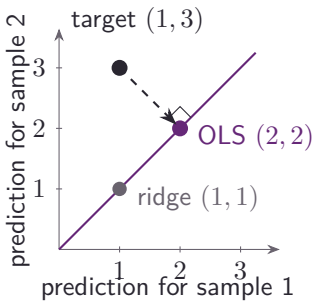


The purple line contains all possible predictions. The target $(1, 3)$ is off the line.

Ordinary least squares (OLS) is a projection

With $\lambda = 0$, the squared fit error is the **distance squared** in this plot:

$$(w-1)^2 + (w-3)^2 = \|(w,w) - (1,3)\|_2^2.$$



OLS: the nearest point is $(2,2)$, with $\widehat{w} = 2$: the **projection** onto the line. The residual is $(2,2) - (1,3) = (1,-1)$. The line's direction is $(1,1)$. Their zero inner product gives the right angle:

$$\Phi^\top(\Phi\widehat{w} - y) = (1,1) \cdot (1,-1) = 0.$$

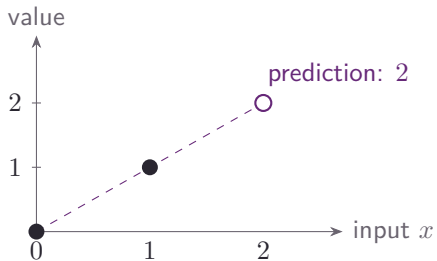
This is the normal equation in the picture. **Ridge** ($\lambda = 2$) picks $(1,1)$: it also pays the weight penalty from page 24.

New example: fit a line using two observed targets

Now the horizontal axis is the **input** x; the vertical axis is a **target or prediction**.

States $x \in \{0, 1, 2\}$; one fixed action; $\phi(x) = (1, x)^\top$; $\lambda = 0$.

$$\phi(x)^\top w = w_1 + w_2 x \qquad \text{(intercept } w_1\text{, slope } w_2\text{)}.$$



- observed labels at $x = 0, 1$

Use only two labels:

$$x = 0,\ y = 0: \quad w_1 = 0,$$
$$x = 1,\ y = 1: \quad w_1 + w_2 = 1.$$

Zero training error:

$$\widehat{w} = (0, 1)^\top, \qquad \widehat{q}(x) = x.$$

No target at $x = 2$ was observed.

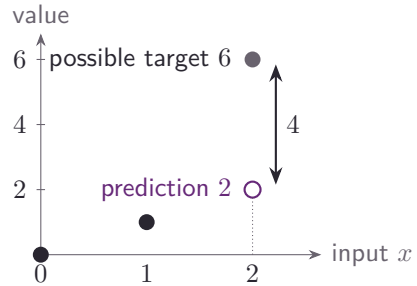
The observed labels cannot rule out another target

The fit is $\widehat{q}(x) = x$. Construct a **possible target** matching both observed labels:

$$f(x) = x + \underbrace{2x(x-1)}_{\text{zero at } x=0,1}.$$

x	Used in fit?	$\widehat{q}(x)$	$f(x)$
0	yes	0	0
1	yes	1	1
2	no	2	6

$f(2) = 2 + 2 \cdot 2 \cdot (2-1) = 6$.
If the target is f, the error at 2 is $6 - 2 = 4$.



If the target is known to have the form $w_1 + w_2 x$, these two noiseless labels suffice. Zero training error alone does not verify that assumption.

We can fit targets; now we must construct them

For fixed design pairs and labels, we have a complete fitting procedure:

$$\{(\phi_i, y_i)\}_{i=1}^n \quad \longrightarrow \quad \Lambda \widehat{w} = \sum_{i=1}^n \phi_i y_i \quad \longrightarrow \quad \widehat{Q}(x,a) = \phi(x,a)^\top \widehat{w}.$$

- The feature map fixes what can be represented.
- Least squares chooses weights from the supplied targets.
- Accuracy at all states still needs justification.

The DP question: optimal values are unknown. Where can useful targets come from?

We will use the known model and a value estimate for the **next stage**.

Least-Squares Value Iteration



Start with the exact backward DP we already know

$Q_t^*(x,a)$: expected return when we force action a now and act optimally afterward.

$$\boxed{Q_t^*(x,a) = r(x,a) + \sum_{x' \in \mathcal{X}} \mathcal{P}(x' \mid x,a) V_{t+1}^*(x').}$$

$$V_t^*(x) = \max_{a \in \mathcal{A}(x)} Q_t^*(x,a), \qquad \pi_t^*(x) \in \operatorname*{argmax}_{a \in \mathcal{A}(x)} Q_t^*(x,a).$$

Start at $V_{H+1}^* = g$, then compute $t = H, H-1, \ldots, 1$.

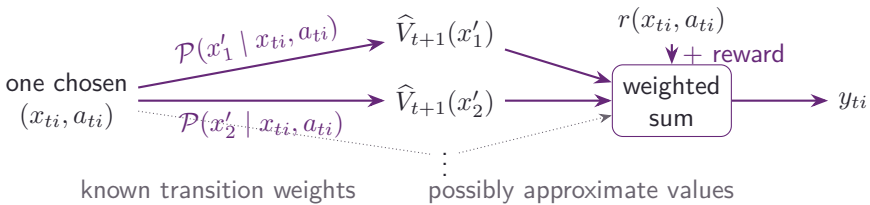


New question: can we replace each full Q_t table by $\phi(x,a)^\top w_t$?

Step 1: compute targets using the next-stage estimate

Suppose $\widehat{V}_{t+1}$ has already been constructed. Choose n_t feasible **design pairs** (x_{ti}, a_{ti}) for target computation. Here t is the stage and i the chosen pair's index; the pairs need not form a trajectory.

$$y_{ti} = r(x_{ti}, a_{ti}) + \sum_{x' \in \mathcal{X}} \mathcal{P}(x' \mid x_{ti}, a_{ti}) \widehat{V}_{t+1}(x').$$



$x'_1, x'_2, \ldots$ are next states. At $t = H$, use $\widehat{V}_{H+1} = g$ exactly.

Step 2: fit one parameter vector for this stage

Write $\phi_{ti} = \phi(x_{ti}, a_{ti}) \in \mathbb{R}^d$. Fit the fixed labels:

$$w_t \in \operatorname*{argmin}_{w \in \mathbb{R}^d} \left\{ \sum_{i=1}^{n_t} (\phi_{ti}^\top w - y_{ti})^2 + \lambda \|w\|_2^2 \right\}.$$

Using the normal equations from the previous section,

$$\Lambda_t := \lambda I + \sum_{i=1}^{n_t} \phi_{ti} \phi_{ti}^\top, \qquad \Lambda_t w_t = \sum_{i=1}^{n_t} \phi_{ti} y_{ti}.$$

If $\lambda > 0$, Λ_t is invertible. If $\lambda = 0$, full column rank of the design matrix suffices.

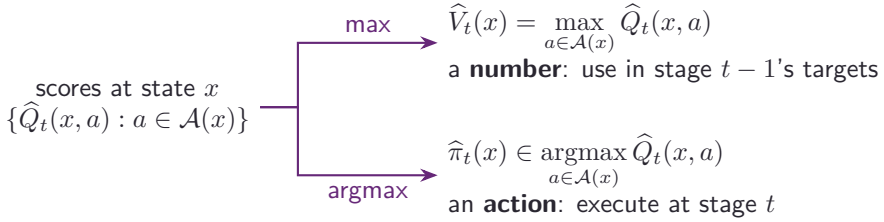


One regression per stage. Shared features; separate stage weights.

Step 3: use the fitted scores to act and to back up

For every feasible input, define

$$\widehat{Q}_t(x,a) = \phi(x,a)^\top w_t.$$

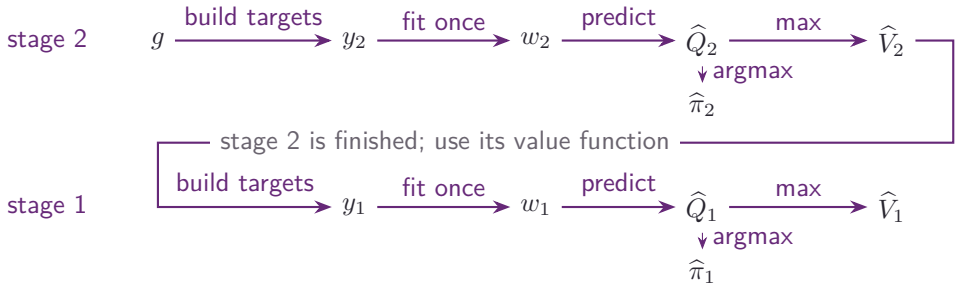


Fix a deterministic tie rule when several actions attain the maximum.

$\widehat{V}_t$ is a fitted prediction. It is **not automatically** the true return of $\widehat{\pi}$.

LSVI: a backward sequence of regression problems

Initialize: $\widehat{V}_{H+1} = g$. Process $t = H, H-1, \ldots, 1$ once each. For $H = 2$, the complete computation is:

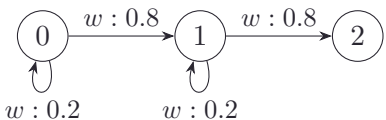


Here $y_t = (y_{t1}, \ldots, y_{tn_t})^\top$ collects the fixed target labels. **Output:** $w_1, \ldots, w_H$ and the stage-dependent policy $\widehat{\pi}$.

Running example: the same job with two work periods

$H = 2$, progress $x \in \{0, 1, 2\}$, initial state $X_1 = 0$.

- Wait q: reward 0, progress unchanged.
- Work w, feasible at $x < 2$: reward -1; advance with probability 0.8, otherwise stay.
- At $x = 2$, only wait is feasible.

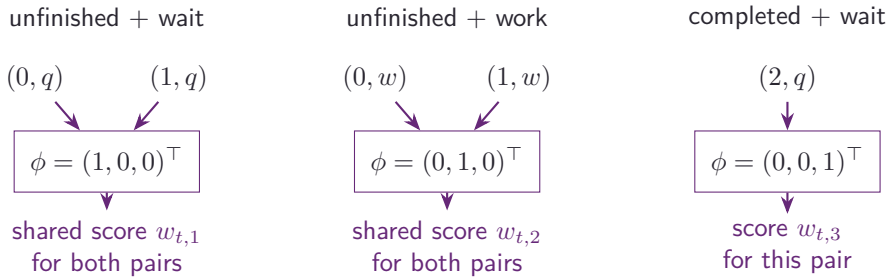


q stays with probability 1 at every state (not drawn).

The terminal payoff at stage 3 is $g = (0, 0, 10)$, in state order $(0, 1, 2)$. The bonus is paid once; work costs are paid when the work is attempted.

A deliberately coarse representation: three shared scores

The same feature map is used at both stages:



$w_{t,j}$ is weight j at stage t; action w still means work.
Five feasible pairs share three parameters; progress 0 and 1 are treated alike.

We retain the original MDP; only its **value representation** is compressed.

Why least squares gives one average per feature group

Use all five feasible pairs once at each stage; set $\lambda = 0$.

$$\begin{matrix}(0,q)\\(0,w)\\(1,q)\\(1,w)\\(2,q)\end{matrix} \qquad \Phi = \begin{bmatrix}1 & 0 & 0\\0 & 1 & 0\\1 & 0 & 0\\0 & 1 & 0\\0 & 0 & 1\end{bmatrix}.$$

Columns: unfinished wait, unfinished work, completed wait.
In $\Phi^\top \Phi$, the diagonal entries $2, 2, 1$ count rows in each group.

Normal equations: add targets in each group.

$$\begin{bmatrix}2 & 0 & 0\\0 & 2 & 0\\0 & 0 & 1\end{bmatrix}\begin{bmatrix}w_{t,1}\\w_{t,2}\\w_{t,3}\end{bmatrix} = \begin{bmatrix}y_{t1}+y_{t3}\\y_{t2}+y_{t4}\\y_{t5}\end{bmatrix}.$$

Divide by the group size:

$$w_{t,1} = \frac{y_{t1}+y_{t3}}{2}, \qquad w_{t,2} = \frac{y_{t2}+y_{t4}}{2},$$

$$w_{t,3} = y_{t5}.$$

Every pair is covered. A group can fit all its targets exactly only if those targets agree.

Stage 2: build targets from the exact terminal payoff

$$y_2(x,a) = r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x,a) g(x'), \quad g = (0,0,10).$$

Pair	Reward + expected terminal payoff	Target
$(0,q)$	$0 + g(0)$	0
$(0,w)$	$-1 + 0.8g(1) + 0.2g(0)$	-1
$(1,q)$	$0 + g(1)$	0
$(1,w)$	$-1 + 0.8g(2) + 0.2g(1)$	7
$(2,q)$	$0 + g(2)$	10

These targets are exact Q_2^* values. Regression may still fail to preserve them.

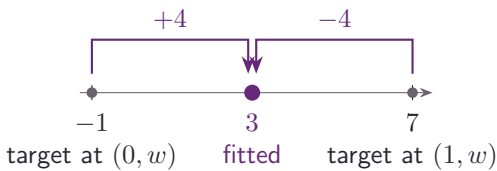
Stage 2: least squares averages incompatible work targets

In row order $(0,q),(0,w),(1,q),(1,w),(2,q)$, we have $y_2 = (0,-1,0,7,10)^\top$.

$$\Phi^\top y_2 = \begin{bmatrix} 0+0 \\ -1+7 \\ 10 \end{bmatrix} = \begin{bmatrix} 0 \\ 6 \\ 10 \end{bmatrix}.$$

$$w_2 = \underbrace{\begin{bmatrix} 1/2 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{(\Phi^\top\Phi)^{-1}} \begin{bmatrix} 0 \\ 6 \\ 10 \end{bmatrix} = \boxed{\begin{bmatrix} 0 \\ 3 \\ 10 \end{bmatrix}}.$$

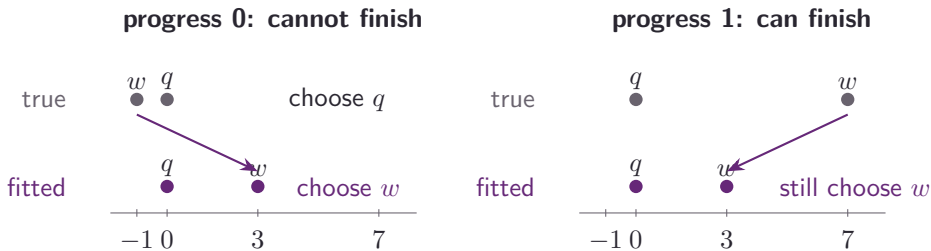
The shared work score



Arrows show fitted score minus target.
One shared score cannot preserve both targets.
Least squares selects their mean.

Stage 2: the representation changes the decision

$w_2 = (0, 3, 10)^\top$. True scores are Q_2^*; fitted scores are $\widehat{Q}_2$. The horizontal axes show **action-value scores**; rows compare true and fitted values.



Thus $\widehat{V}_2 = (3, 3, 10)$: work at 0 and 1; at 2 only wait is feasible, with score 10.

The model was exact. The features erased a decision-relevant difference.

Quiz: complete stage 1 of LSVI

Stage 2 is finished: $\widehat{V}_2 = (3, 3, 10)$, in state order $(0, 1, 2)$.

q: reward 0, stay. w: reward -1, advance by 1 with probability 0.8, otherwise stay. Work is feasible only at $x < 2$; at $x = 2$, only q is feasible.

$y_1(x, a) = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a)\widehat{V}_2(x')$.

1. Build the five targets.

Pair	$\phi(x,a)^\top$	$y_1(x,a)$
$(0, q)$	$(1, 0, 0)$	___
$(0, w)$	$(0, 1, 0)$	___
$(1, q)$	$(1, 0, 0)$	___
$(1, w)$	$(0, 1, 0)$	___
$(2, q)$	$(0, 0, 1)$	___

2. Fit w_1 by least squares.
Use all five pairs once; $\lambda = 0$. You may average targets within feature groups.

3. Maximize the fitted scores.
$\widehat{Q}_1(x, a) = \phi(x, a)^\top w_1$.
Report $\widehat{V}_1(x)$ and $\widehat{\pi}_1(x)$ for $x = 0, 1, 2$.

Work individually.

Quiz answer: build the five stage-1 targets

Now use $\widehat{V}_2 = (3, 3, 10)$, not g:

$$y_1(x, a) = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a)\widehat{V}_2(x').$$

Pair	Reward + expected fitted next-stage value	Target
$(0, q)$	$0 + 3$	3
$(0, w)$	$-1 + 0.8(3) + 0.2(3)$	2
$(1, q)$	$0 + 3$	3
$(1, w)$	$-1 + 0.8(10) + 0.2(3)$	7.6
$(2, q)$	$0 + 10$	10

The transition average is still exact; its input $\widehat{V}_2$ is not exact.

Quiz answer: fit the weights, then maximize fitted scores

The feature matrix has not changed. Average targets within each group:

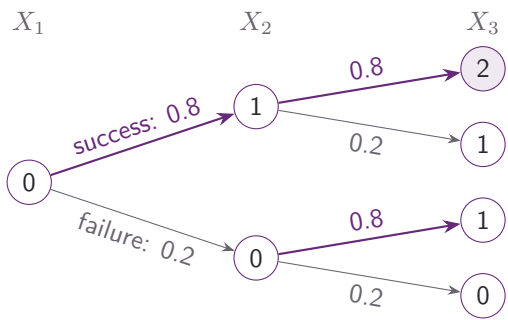
$$w_1 = \begin{bmatrix} (3+3)/2 \\ (2+7.6)/2 \\ 10 \end{bmatrix} = \begin{bmatrix} 3 \\ 4.8 \\ 10 \end{bmatrix}.$$

x	$\widehat{Q}_1(x,q)$	$\widehat{Q}_1(x,w)$	$\widehat{V}_1(x)$	$\widehat{\pi}_1(x)$
0	3	4.8	4.8	work
1	3	4.8	4.8	work
2	10	infeasible	10	wait

Output policy: work whenever unfinished, at both stages. This is the rollout rule we evaluated in Lecture 5.

Check: is the predicted value 4.8 the policy's return?

Start at progress 0 and work twice. Each actual reward is $R_t = -1$; each attempt independently succeeds with probability 0.8. The terminal bonus is 10 only at progress 2.

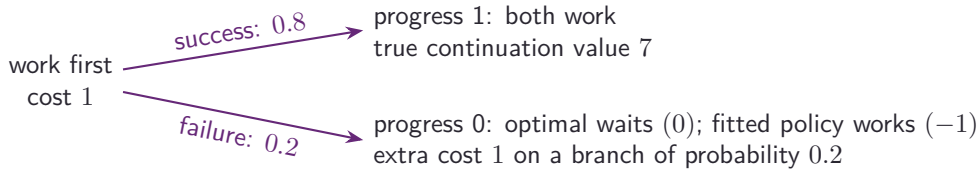


probability	total return
$0.8^2 = 0.64$	$-2 + 10 = 8$
$0.8(0.2) = 0.16$	-2
$0.2(0.8) = 0.16$	-2
$0.2^2 = 0.04$	-2

Compute $V_1^{\widehat{\pi}}(0)$ by averaging the four total returns above. Why can it differ from the fitted prediction $\widehat{V}_1(0) = 4.8$?

Answer: prediction, optimal value, and policy return differ

Learned policy: $V_1^{\widehat{\pi}}(0) = 0.64(8) + 0.36(-2) = \boxed{4.4}$. Compare with optimal continuation. At stage 2, the policies differ only after failure:



$$Q_1^*(0,q) = 0 + V_2^*(0) = 0,$$
$$Q_1^*(0,w) = -1 + 0.8(7) + 0.2(0) = 4.6.$$

$$V_1^*(0) = \max\{0, 4.6\} = \boxed{4.6}.$$

In this example: $\underbrace{\widehat{V}_1(0) = 4.8}_{\text{fitted prediction}}$, $\underbrace{V_1^*(0) = 4.6}_{\text{best achievable return}}$, $\underbrace{V_1^{\widehat{\pi}}(0) = 4.4}_{\text{learned policy's return}}$.

A fitted prediction need not be achievable by any policy.

One independent coordinate per pair recovers tabular DP

As on the one-hot slides, use $N = \sum_{x \in \mathcal{X}} |\mathcal{A}(x)|$ coordinates: one per feasible pair. Fit **every pair once**, in the same order as the coordinates; set $\lambda = 0$.
Job: $N = 5$, hence $\Phi = I_5$.

$$\begin{matrix}(0,q)\\(0,w)\\(1,q)\\(1,w)\\(2,q)\end{matrix} \qquad \Phi = \begin{bmatrix}1&0&0&0&0\\0&1&0&0&0\\0&0&1&0&0\\0&0&0&1&0\\0&0&0&0&1\end{bmatrix}.$$

Work at 0 and work at 1 now have separate weights.

For any number of pairs: $\Phi = I_N$.

$$w_t = (I_N^\top I_N)^{-1} I_N^\top y_t = y_t.$$

So every target is preserved:

$$\widehat{Q}_t(x,a) = y_t(x,a).$$

One-hot uses $d = N$: no parameter saving.

Starting from $\widehat{V}_{H+1} = g$, each backup is exact and each fit preserves it. Backward induction gives $\widehat{Q}_t = Q_t^*$ and $\widehat{V}_t = V_t^*$.

What has become cheaper, and what still costs work?

Potential savings

- Store H vectors of length d.
- Fit at selected design pairs.
- Predict at inputs not used for fitting.

Remaining work

- Compute each model expectation.
- Solve the regression problems.
- Maximize over feasible actions.

Known model does not mean cheap expectation. Large transition supports can still make target construction expensive.

Feature quality and design coverage determine whether the savings are useful.

What Does Approximation Change?



A residual compares the fitted score with its own target

Define the target at **every** feasible pair, including pairs not used for fitting:

$$y_t(x,a) := r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x,a)\widehat{V}_{t+1}(x').$$

Uniform Bellman-fit residual:

$$\boxed{\varepsilon_t := \max_{x,\, a \in \mathcal{A}(x)} |\widehat{Q}_t(x,a) - y_t(x,a)|.}$$

"Uniform" means one bound valid for **all feasible pairs**.

Use absolute error, not squared loss. Compare with the backup of $\widehat{V}_{t+1}$, not directly with Q_t^*.

Our full-data stage-2 fit:

Pair	y_2	$\widehat{Q}_2$	$\lvert\widehat{Q}_2 - y_2\rvert$
$(0,q)$	0	0	0
$(0,w)$	-1	3	4
$(1,q)$	0	0	0
$(1,w)$	7	3	4
$(2,q)$	10	10	0

$$\varepsilon_2 = \max\{0,4,0,4,0\} = 4.$$

Zero training error can hide a large error elsewhere

Stage 2 of the job: q waits; w costs 1 and advances with probability 0.8. Terminal payoffs are $g(0) = g(1) = 0$, $g(2) = 10$.

Change the data and refit: Use these three pairs instead of all five; $\lambda = 0$.

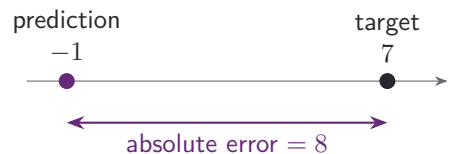
Pair	Feature	Target
$(0, q)$	$(1, 0, 0)^\top$	0
$(0, w)$	$(0, 1, 0)^\top$	-1
$(2, q)$	$(0, 0, 1)^\top$	10

$$\Phi = I_3, \qquad w_2 = (0, -1, 10)^\top.$$

Every training target is fitted exactly.

Query the omitted pair $(1, w)$:

$$\underbrace{\phi(1, w) = \phi(0, w)}_{\text{the same unfinished-work feature}} \qquad = (0, 1, 0)^\top.$$



$$y_2(1, w) = -1 + 0.8(10) + 0.2(0) = 7.$$

Full column rank gives a unique fit; it does not guarantee accuracy at omitted pairs.

A fitted error and a future-value error both enter

Let $e_t := \max_x |\widehat{V}_t(x) - V_t^*(x)|$: the worst state-value error at stage t. For every feasible (x, a), insert the target $y_t(x, a)$:

$$
\begin{aligned}
|\widehat{Q}_t(x,a) - Q_t^*(x,a)| &\le \underbrace{|\widehat{Q}_t(x,a) - y_t(x,a)|}_{\text{current fit error}} + \underbrace{|y_t(x,a) - Q_t^*(x,a)|}_{\text{inherited value error}} \\
&\le \varepsilon_t + \left| \sum_{x'} \mathcal{P}(x' \mid x, a)[\widehat{V}_{t+1}(x') - V_{t+1}^*(x')] \right| \\
&\le \varepsilon_t + \sum_{x'} \mathcal{P}(x' \mid x, a)|\widehat{V}_{t+1}(x') - V_{t+1}^*(x')| \\
&\le \varepsilon_t + \sum_{x'} \mathcal{P}(x' \mid x, a) e_{t+1} \\
&= \varepsilon_t + e_{t+1}.
\end{aligned}
$$

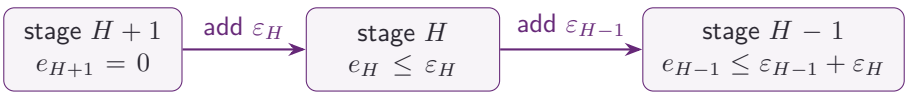
Rewards cancel; transition probabilities are nonnegative and sum to one.

The terminal boundary turns the recursion into a finite sum

At each state x, maximize over its feasible actions:

$$|\widehat{V}_t(x) - V_t^*(x)| = |\max_a \widehat{Q}_t(x,a) - \max_a Q_t^*(x,a)|$$
$$\leq \max_a |\widehat{Q}_t(x,a) - Q_t^*(x,a)| \leq \varepsilon_t + e_{t+1}.$$

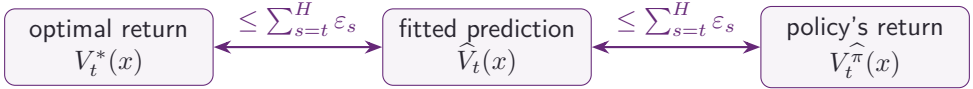
For every x, the bound is the same: $e_t \leq \varepsilon_t + e_{t+1}$. At the terminal boundary, both values equal g, so $e_{H+1} = 0$.



$$\max_x |\widehat{V}_t(x) - V_t^*(x)| \leq \sum_{s=t}^{H} \varepsilon_s.$$

Two error comparisons bound the policy's loss

At state x, both absolute differences have the same bound:



Insert $\widehat{V}_t(x)$ between the two true returns and use the triangle inequality:

$$0 \leq V_t^*(x) - V_t^{\widehat{\pi}}(x) \leq |V_t^*(x) - \widehat{V}_t(x)| + |\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x)|$$
$$\leq \boxed{2 \sum_{s=t}^{H} \varepsilon_s}.$$

The fitted value need not lie between the returns. Assume exact terminal g and residual bounds over **all pairs**. Policy comparison: see the optional proof.

Bellman completeness: a backup stays representable

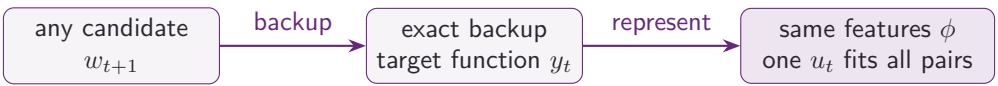
At each stage $t < H$, take **any** next-stage weights $w_{t+1} \in \mathbb{R}^d$:

$$\widehat{V}_{t+1}(x') = \max_{b \in \mathcal{A}(x')} \phi(x', b)^\top w_{t+1},$$

$$y_t(x, a) = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a) \widehat{V}_{t+1}(x').$$

Bellman completeness: there is **one** vector $u_t \in \mathbb{R}^d$ such that

$$\boxed{y_t(x, a) = \phi(x, a)^\top u_t \quad \text{for every feasible } (x, a).}$$

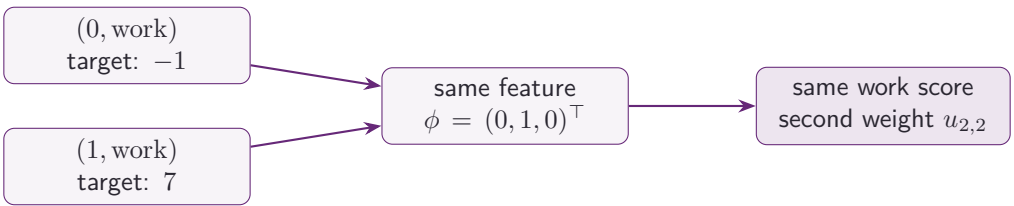


Terminal boundary: use $\widehat{V}_{H+1} = g$; require y_H to be representable too. The earlier residual bounds did not assume this closure property.

The job's shared work feature cannot fit both targets

Job: two stages with the same rules. Work costs 1, advances with probability 0.8, otherwise stays. At progress 2, only wait is feasible. $g(0) = g(1) = 0$, $g(2) = 10$.

$$y_2(x, w) = -1 + 0.8\, g(x+1) + 0.2\, g(x), \qquad x = 0, 1.$$

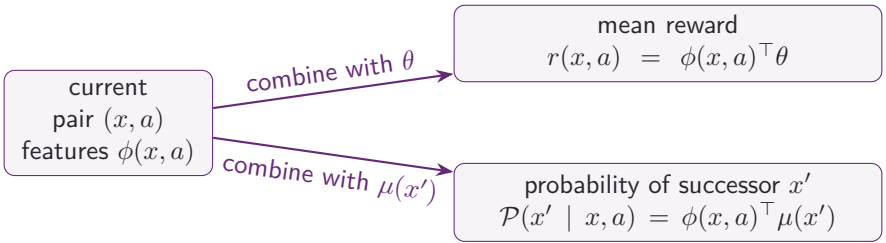


One weight would have to equal both -1 and 7: **no exact representation exists**.

Candidate weights $(0, 0, 10)^\top$ give $\widehat{V}_2 = g$: both unfinished actions score 0, completed wait scores 10. Their stage-1 backup also fails.

Linear MDP: the same features also represent the model

Assume model coefficients $\theta \in \mathbb{R}^d$ and $\mu(x') \in \mathbb{R}^d$ for each successor x'. The **same known feature map** describes rewards and transitions:



The resulting probabilities must be nonnegative and sum to one for each (x, a). Individual coordinate functions μ_j need not be probability distributions.

Our coarse job features fail this assumption: they identify the two work pairs, but $\mathcal{P}(2 \mid 0, w) = 0$ and $\mathcal{P}(2 \mid 1, w) = 0.8$.

A linear MDP guarantees Bellman completeness

For **any** next-stage value function $V : \mathcal{X} \to \mathbb{R}$, substitute the model formulas:

$$r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x,a) V(x') = \phi(x,a)^\top \theta + \sum_{x'} \phi(x,a)^\top \mu(x') V(x')$$

$$= \phi(x,a)^\top \underbrace{\left[\theta + \sum_{x'} \mu(x') V(x')\right]}_{\text{one coefficient vector, independent of } (x,a)} .$$

The factor $\phi(x,a)$ is independent of the successor x', so it leaves the sum.

This applies to both $V(x') = \max_{b \in \mathcal{A}(x')} \phi(x',b)^\top w$ and the terminal $V = g$.

linear MDP with ϕ → (sufficient condition) → Bellman completeness for ϕ

Coverage, sampling noise, and ridge can still cause fitting error.

Next: replace an exact model target by an observed target

Known model, as today:

$$y = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a) \widehat{V}_{t+1}(x').$$

A fresh sample: $\mathbb{E}[R \mid x, a] = r(x, a)$, $X' \sim \mathcal{P}(\cdot \mid x, a)$.

$$\widetilde{y} = R + \widehat{V}_{t+1}(X').$$

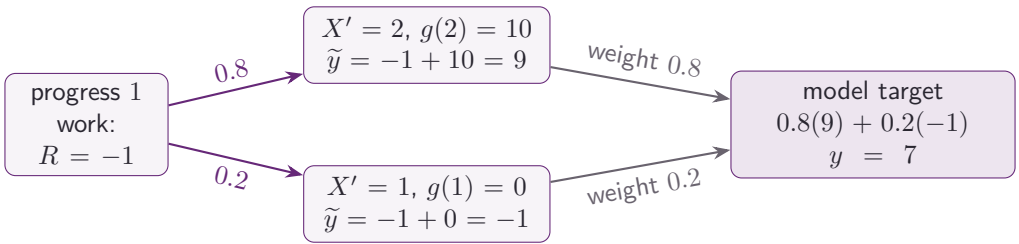
For a fixed $\widehat{V}_{t+1}$ independent of this fresh draw,

$$\mathbb{E}[\widetilde{y} \mid x, a] = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a) \widehat{V}_{t+1}(x') = y.$$

Sampling error now joins approximation error. Reusing data that fitted $\widehat{V}_{t+1}$ requires additional statistical care.

The exact target averages the possible sampled labels

At stage 2 of the job: progress $x = 1$, choose work w, reward $R = -1$. The next stage is terminal: $\widehat{V}_3 = g$, with $g(1) = 0$, $g(2) = 10$.



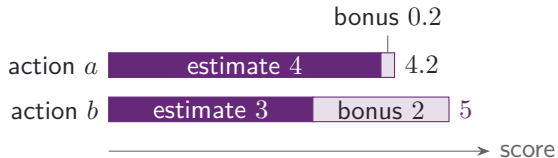
One fresh transition gives either 9 or -1, not 7. The exact model averages both outcomes; the sampled label has mean 7.

Later: uncertainty can change which action we try

Actions determine which data we collect. LSVI with **upper confidence bounds** (LSVI-UCB) adds a bonus to encourage exploration.

$$\underbrace{\phi(x,a)^\top w_t}_{\text{fitted estimate}} + \underbrace{\beta\sqrt{\phi(x,a)^\top \Lambda_t^{-1}\phi(x,a)}}_{\text{uncertainty bonus}}, \quad \Lambda_t = \lambda I_d + \sum_i \phi_i \phi_i^\top, \quad \lambda, \beta > 0.$$

ϕ_i: collected stage-t features. β: bonus scale, calibrated in the later lecture. Choose the largest estimate-plus-bonus score; pale bars show the bonus.



Illustrative scores for two feasible actions.

Less data along a feature direction gives a larger bonus.
Here b wins: $3 + 2 > 4 + 0.2$.
The environment does not pay this bonus.

The main idea: back up, fit, and act

1. **Represent:** features share parameters across state–action pairs.
2. **Fit:** least squares matches fixed labels; ridge makes the linear system invertible.
3. **Back up:** LSVI constructs labels from the next-stage fitted value and works backward from g.
4. **Check:** a fitted score is not a policy return. Representation, coverage, and model structure matter.

Next in the course: practice modeling and DP in Lab 1, then learn values from sampled experience. We will return to backward regression when studying exploration in linear MDPs.

Optional:
Proof Details



Why maximizing does not amplify a uniform score error

Let u_a and v_a be two real-valued scores for each feasible action a. Define $D = \max_a |u_a - v_a|$, their largest absolute difference. For every a,

$$|u_a - v_a| \le D \quad \Longrightarrow \quad u_a - v_a \le D \quad \Longrightarrow \quad u_a \le v_a + D \le \max_b v_b + D.$$

The right-hand side is the same for all a, so

$$\max_a u_a - \max_b v_b \le D.$$

Interchange u and v to obtain the opposite inequality. Therefore

$$|\max_a u_a - \max_a v_a| \le \max_a |u_a - v_a|.$$

The maximizing actions need not be the same.

The policy comparison uses the same error recursion

Let $f_t := \max_x |\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x)|$. At state x, take $a = \widehat{\pi}_t(x)$, so $\widehat{V}_t(x) = \widehat{Q}_t(x, a)$. The policy Bellman equation and the definition of y_t give

$$\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x) = \widehat{Q}_t(x, a) - y_t(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a)[\widehat{V}_{t+1}(x') - V_{t+1}^{\widehat{\pi}}(x')].$$

Now take absolute values, then bound each next-state difference:

$$|\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x)| \le |\widehat{Q}_t(x, a) - y_t(x, a)| + \sum_{x'} \mathcal{P}(x' \mid x, a)|\widehat{V}_{t+1}(x') - V_{t+1}^{\widehat{\pi}}(x')|$$

$$\le \varepsilon_t + \sum_{x'} \mathcal{P}(x' \mid x, a) f_{t+1} = \varepsilon_t + f_{t+1}.$$

Probabilities sum to one. The bound is the same for every x, so

$$\boxed{f_t \le \varepsilon_t + f_{t+1}}, \qquad f_{H+1} = 0 \quad \text{because both terminal values equal } g.$$

Combine two finite-sum bounds to control policy loss

Backward induction gives

$$\max_x |\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x)| \le \sum_{s=t}^{H} \varepsilon_s.$$

For any state x, split the policy loss at the fitted value:

$$\begin{aligned} V_t^*(x) - V_t^{\widehat{\pi}}(x) &= [V_t^*(x) - \widehat{V}_t(x)] + [\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x)] \\ &\le |V_t^*(x) - \widehat{V}_t(x)| + |\widehat{V}_t(x) - V_t^{\widehat{\pi}}(x)| \\ &\le 2 \sum_{s=t}^{H} \varepsilon_s. \end{aligned}$$

Nonnegativity follows because V_t^* is optimal over all admissible policies. The fitted value may lie above or below either true value.